

DIVIDE AND CONQUER

PAGE NO.

DATE

It contains 3 steps :-

1. Divide
2. Conquer
3. ~~split~~ combine
3. combine

Algorithm :-

Algorithm D and C (P) ^{problem}

{

If P is small return S(P)

else

{

divide P in to smaller instances $P_1, P_2, \dots, P_K$.

Apply D and C to each Subproblem.

return combine (D and C (P_1), D and C (P_2), ...
D and C (P_K))

}

}

$$T(n) = \begin{cases} g(n) & n = \text{small} \\ T(n_1) + T(n_2) + \dots + T(n_k) & \end{cases}$$

Time complexity :-

$$T(n) = \begin{cases} T(1) & \end{cases}$$

 $n=1$

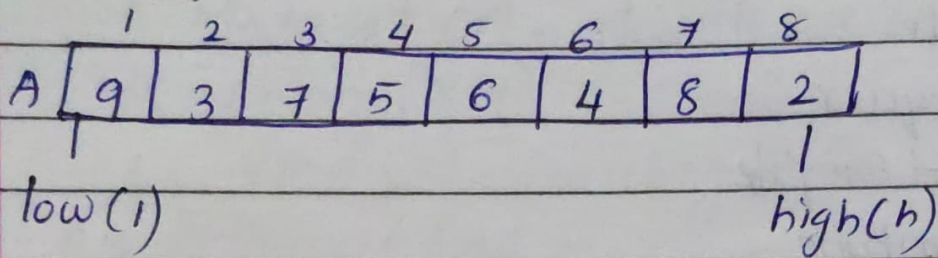
Base Condition

$$\begin{cases} aT(n/b) + f(n) & \end{cases}$$

 $n > 1$

Recursive Condition.

MERGE SORT



Algorithm :-

void partition (int a[], low, high)

{
 if (low < high)

{

 mid = (low + high) / 2

 partition (a, low, mid)

 partition (a, mid + 1, high)

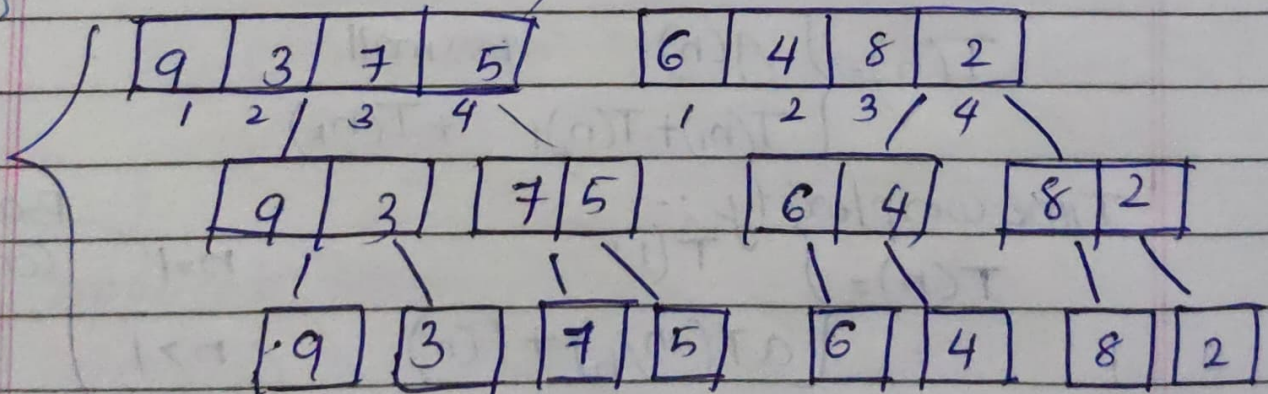
 Merge (low, mid, high)

 }

}

$$\text{mid} = \frac{1+8}{2} = \frac{9}{2} = 4.5$$

Dividing



Merge Algorithm:-

Merge (int a[], low, mid, high)

{

h = i = low

j = mid + 1

while (h ≤ mid) && (j ≤ high)

{

if (a[h] ≤ a[j])

{

h[i] = a[h]

h++

}

else

{

h[j] = a[j]

j++

}

i++

}

for (k = j to high)

{

h[i] = a[k]

h++

}

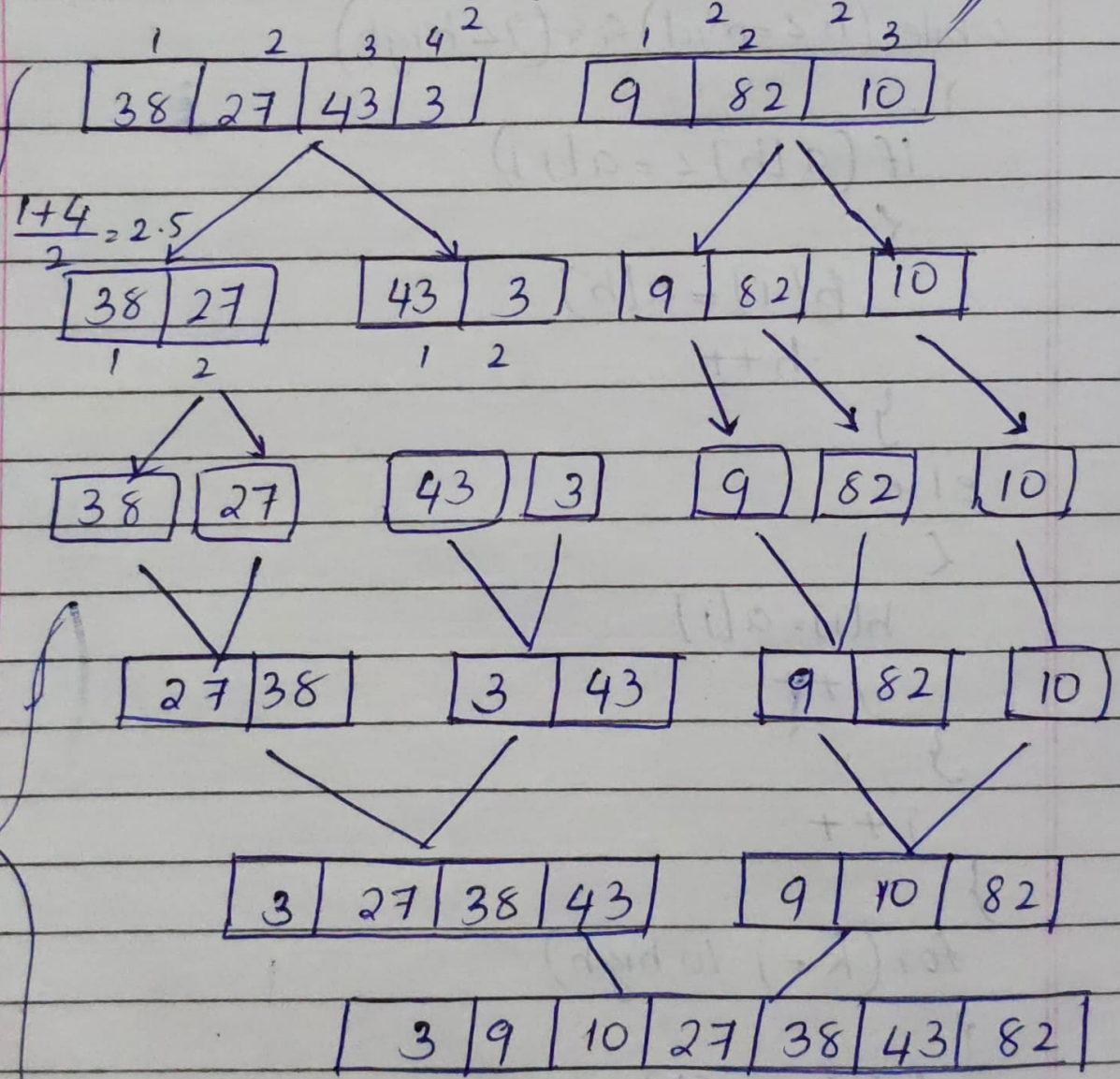
eg:-

1	2	3	4	5	6	7
38	27	43	3	9	82	10

low high

$$\text{mid} = \text{low} + \text{high} = \frac{1+7}{2} = \frac{8}{2} = 4$$

Dividing



Merging Algorithm

Time complexity:-

$$T(n) = \begin{cases} a & n=1 \\ 2T(n/2) + cn & n>1 \end{cases}$$

Solving using Iteration method.

Substitution Method:-

$$T(n) = 2T(n/2) + cn \quad \text{--- (1)}$$

$$T(n/2) = 2T(n/4) + cn/2$$

$$T(n/2) = 2T(n/4) + cn/2 \quad \text{--- (2)}$$

(2) in (1) $\Rightarrow$

$$T(n) = 2[2T(n/4) + cn/2] + cn$$

$$= 4T(n/4) + 2cn/2 + cn$$

$$= 4T(n/4) + T \cdot cn + cn$$

$$T(n) = 2[2T(n/4) + cn/2] + cn$$

$$= 4T(n/4) + cn + cn$$

$$= 2^2 T(n/2^2) + 2cn$$

$$T(n) = 2^k T(n/2^k) + Kcn$$

$$n/2^k = 1 \Rightarrow n = 2^k \Rightarrow \log n = \log 2^k$$

$$\log n = k \log_2^2$$

$$\therefore \log n = k$$

$$T(n) = 2^{\log n} T\left(\frac{n}{2}\right) + \log n \cdot cn$$

$$= 2^{\log n} \cdot T(1) + \log n \cdot cn$$

$$= 2^{\log n} \cdot T(1) + cn \log n$$

$$= 2^{\log n} \cdot a + cn \log n$$

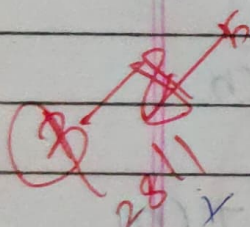
$$= n \log_2^2 \cdot a + cn \cdot \log n$$

$$= n \cdot a + cn \cdot \log n \quad \text{Here } n \log n \gg n$$

$$= an + cn \cdot \log n$$

$$= n + n \log n$$

$$= O(n \log n)$$



STRASSEN'S MATRIX MULTIPLICATION

Matrix Multiplication :-

$\Rightarrow 8T(n/2) + n^2 \leftarrow$ time complexity in normal cases :-

$$T(n) = 8T(n/2) + n^2 \quad \text{--- (1)}$$

$$T(n/2) = 8T(n/4) + (n/2)^2$$

$$= 8T(n/4) + \frac{n^2}{4} \quad \text{--- (2)}$$

$$= 8T(n/4) + \frac{n^2}{4}$$

$$\frac{n}{2} \times \frac{n}{2} = \frac{n^2}{4}$$

eqn (2) in (1) $\Rightarrow$

$$T(n) = 8 \left[8T\left(\frac{n}{4}\right) + \frac{n^2}{4} \right] + n^2$$

$$= 64T\left(\frac{n}{4}\right) + 8 \cdot \frac{n^2}{4} + n^2$$

$$= 64T\left(\frac{n}{4}\right) + 2n^2 + n^2$$

$$= 64T\left(\frac{n}{4}\right) + 3n^2$$

$$= 64T\left(\frac{n}{4}\right) + 3n^2$$

$$T(n) = 2^k T\left(\frac{n}{2^k}\right) +$$

$$\Rightarrow T(n) = 8T\left(\frac{n}{2}\right) + n^2$$

$$g(n) = n \log_a b$$

$$= n \log_2 8$$

STRASSEN'S MATRIX MULTIPLICATION ALGO...

Step 1: A and B are the matrices with dimension $n \times n$.

Step 2: If n is not a power of 2, then enough rows and columns of 0's can be added to both A and B. So that the resulting dimensions are the power of 2.

Step 3: partition A and B into 4 square matrices of size $\frac{n}{2} \times \frac{n}{2}$.

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \times \begin{bmatrix} e & f \\ g & h \end{bmatrix}$$

A B

Step 4: Compute 7 $n/2 \times n/2$ matrices.

$$P_1 = a(f-h)$$

$$P_2 = h(a+b)$$

$$P_3 = e(c+d)$$

$$P_4 = d(g-e)$$

$$P_5 = (a+d)(e+h)$$

$$P_6 = (b-d)(g+h)$$

$$P_7 = (a-c)(e+f)$$

Step 5: Then compute

$$C = \begin{bmatrix} C_1 & C_2 \\ C_3 & C_4 \end{bmatrix}$$

$$C_1 = P_4 + P_5 + P_6 - P_2$$

$$C_2 = P_1 + P_2$$

$$C_3 = P_3 + P_4$$

$$C_4 = P_1 - P_3 + P_5 - P_7$$

Tym Complexity: $- 7 T(n/2) + n^2$ addition & subtraction

Q. Multiply the following matrices using strassen's matrix multiplication?

$$A = \begin{bmatrix} 6 & 8 \\ 9 & 7 \end{bmatrix} \quad B = \begin{bmatrix} 2 & 5 \\ 3 & 6 \end{bmatrix}$$

$$\begin{array}{r} 14 \\ 6 \\ 2 \times 6 \\ \hline 84 \\ 1 \times 16 \\ 2 \\ \hline 18 \end{array}$$

ans:-

$$P_1 = a(f-h) = 6(5-6) = 6 \times -1 = -6 //$$

$$P_2 = b(g+h) = 6(6+8) = 6 \times 14 = 84 //$$

$$P_3 = e(c+d) = 2(9+7) = 32 //$$

$$P_4 = d(g-e) = 7(3-2) = 7 //$$

$$P_5 = (a+d)(e+h) = (6+7)(2+6)$$

$$= 13 \times 8$$

$$= 104 //$$

$$P_6 = (b-d)(g+h) = (8-7)(3+6)$$

$$= 1 \times 9$$

$$= 9 //$$

$$P_7 = (a-c)(e+f) = (6-9)(2+5)$$

$$= -3 \times 7$$

$$= -21 //$$

$$C_1 = P_4 + P_5 + P_6 - P_2$$

$$= 7 + 104 + 9 - 84$$

$$= 36 //$$

$$C_2 = P_1 + P_2$$

$$= -6 + 84$$

$$= 78 //$$

$$C_3 = P_3 + P_4 = 32 + 7 \\ = 39 //$$

$$C_4 = P_1 - P_3 + P_5 - P_7 \\ = -6 - 32 + 104 - -21 \\ = 87 //$$

$$C = \begin{pmatrix} 36 & 78 \\ 39 & 87 \end{pmatrix}$$

GREEDY STRATEGY

Algorithm Greedy(a, n) Arraysize

// a [1:n] contains n inputs

Solution = $\emptyset$

for (i=1 to n do

{

x = select(a)

if feasible (Solution, x)

Solution = Union (Solution, x)

}

return Solution

}

KNAPSACK PROBLEM

- divided into two :- fractional and zero one.

Algorithm :-

1. Compute the profit ^{by weight} ~~per value~~ ratio of each item.
2. Sort all the items in decreasing order of their profit by weight ratio.
3. Start filling the knapsack by putting the items into ^{it} one by one.
- 4.

Algorithm GreedyKnapsack (m, n)

{

n-objects, m-Size of knapsack, p-profit

for (i=1 to n) do $x[i] = 0.0$

$U = m$

for (i=1 to n) do

{

if ($w[i] > 0$) then

SA
PAGE NO.
DATE / /

ABC
PAGE NO.
DATE / /

Step 1: Find profit by weight for each objects.

Step 2: Since we have to select completely or partially so we sort the profit by weight.

Step 3: The object with the highest profit by weight is selected first.

Step 4: Mark the object with 1, if it completely select or fractions part if it is not selected completely.

Step 5: While we select a particular object deduct the knapsack size by its particular object size.

Step 6: Repeat step 4 and 5.

Step 7: Note the final fractional part and count that object in knapsack.

Step 8: Find the Total weight and profit earned.

Q. Eg: For the given set of items and knapsack size 60 kg. find optimal solution for fractional knapsack problem?

ans:- $\{w_1, w_2, w_3, w_4, w_5\} = \{5, 10, 15, 22, 25\}$

$\{p_1, p_2, p_3, p_4, p_5\} = \{30, 40, 45, 77, 90\}$

ans:- $I_1 = \frac{30}{5} = 6 //$
 $I_2 = \frac{40}{10} = 4 //$
 $I_3 = \frac{45}{15} = 3 //$
 $I_4 = \frac{77}{22} = 3.5 //$
 $I_5 = \frac{90}{25} = 3.6 //$

Item	profit	weight	profit/weight
1 ₁	30	5	6
1 ₂	40	10	4
1 ₃	45	15	3
1 ₄	77	22	3.5
1 ₅	90	25	3.6

Step 2:-

Item	profit	weight	pro/weight	
1 ₁	30	5	6	
1 ₂	40	10	4	
1 ₅	90	25	3.6	
1 ₄	77	22	3.5	
1 ₃	45	15	3	
Remaining weight		$\times$ whether it is earned	P _i X _i	
60 - 5 = 55		1	1 x 30 = 30	
55 - 10 = 45		1	1 x 40 = 40	
45 - 25 = 20		1	1 x 90 = 90	
20 - 22 = -2		20/22	$\frac{20}{22} \times 77 = 70$	
60 - 15 = 0		0	0	

Total profit = $\sum p_i x_i = 230 //$

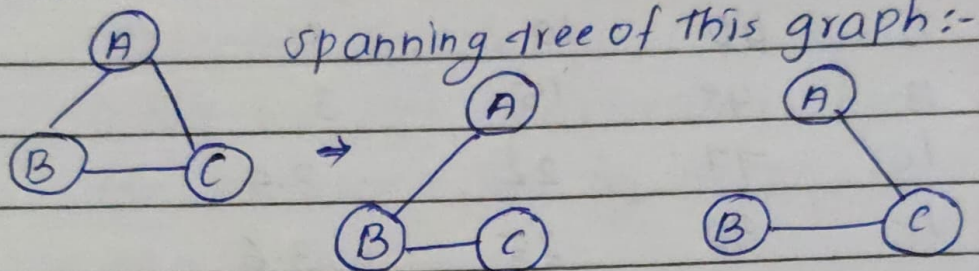
Total weight = 60 kg.

= $\sum w_i x_i$

= $1 \times 5 + 1 \times 10 + 1 \times 25 + \frac{20}{22} \times 22 + 0 \times 15$

Items included in Knapsack = $\{1, 12, 15, 14\}$

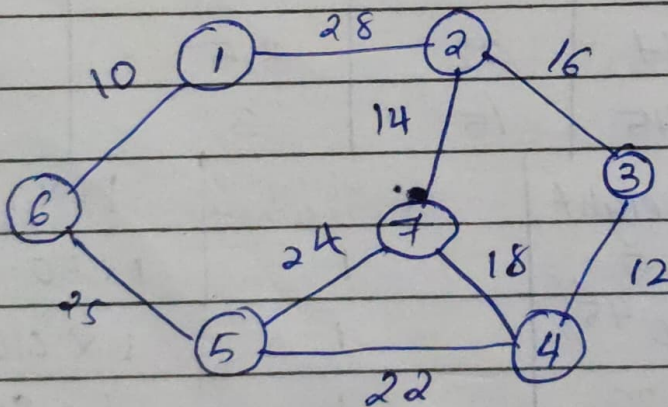
Q. SPANNING TREE



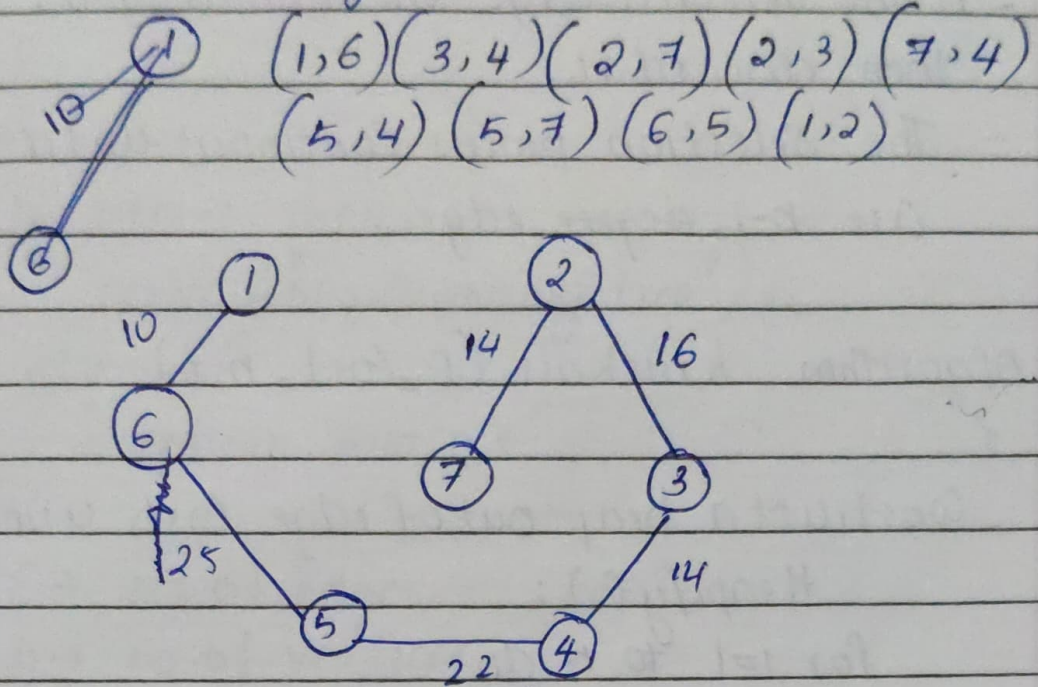
no. of Spanning tree = no. of total edges - 1

Imp KRUSKAL'S ALGORITHM :-

Q. Consider the minimum Spanning Tree (MST) for the given graph using Kruskal's Algorithm.



sorting order :-



- while drawing $(7, 4)$, it makes a cycle, so Reject it.
- Reject $(5, 7)$
- Reject $(1, 2)$

Minimum Cost = $10 + 25 + 22 + 14 + 16 + 14$
 $= 99$

Algorithm :-

- ⇒ - Build the Spanning tree by adding edges one by one into a growing spanning tree.
- In each iteration it finds an edge which has least weight and add to the growing spanning tree.
 - Edges of the graph are considered in the increasing order of cost.

- If the selected edge will form a cycle, then discard it.
- The selection process continues until there are $n-1$ edges.

Algorithm $\text{kruskals}(E, \text{cost}, n, t)$

{

Construct a heap out of edge costs using

$\text{Heapify}()$;

for $i=1$ to n do

$\text{parent}[i] = -1$;

$j=0$

$\text{mincost} = 0.0$

while ($i \leq n-1$) and (heap not empty) do

{

Delete a minimum cost edge (u, v) from the heap and reheapify using $\text{Adjust}()$;

$J = \text{find}(u)$;

$k = \text{find}(v)$;

If $J \neq k$ then

{

$i = i+1$

$t[b1] = u$ $t[1,2] = v$;

$\text{mincost} = \text{mincost} + \text{cost}(u, v)$;

```

    Union (j, k) ;
}
}
if i+n-1 then
    write ("No spanning tree");
else
    return mincost
}

```

$E \rightarrow$ set of edges

$n \rightarrow$ no. of vertices

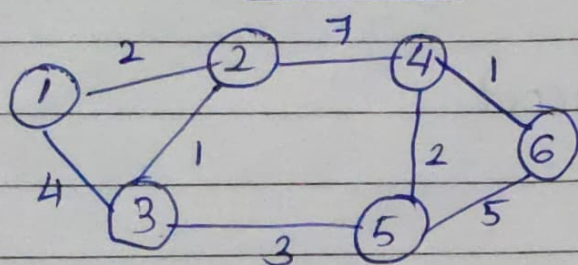
$t \rightarrow$ Set of edges in the minimum cost spanning tree.
The final cost is returned.

- **Heapify()**: Construct a minheap based on the edge cost of G .
- **Adjust()**: reconstruct a minheap if there is a deletion occurs.
- **find()**: Returns the set no. of that particular vertex j and k are the set no. of vertex u and v respectively.
- if $J=K$ vertex u and v are belongs to the same set. Inclusion of (u, v) should definitely form cycle. So discard it.

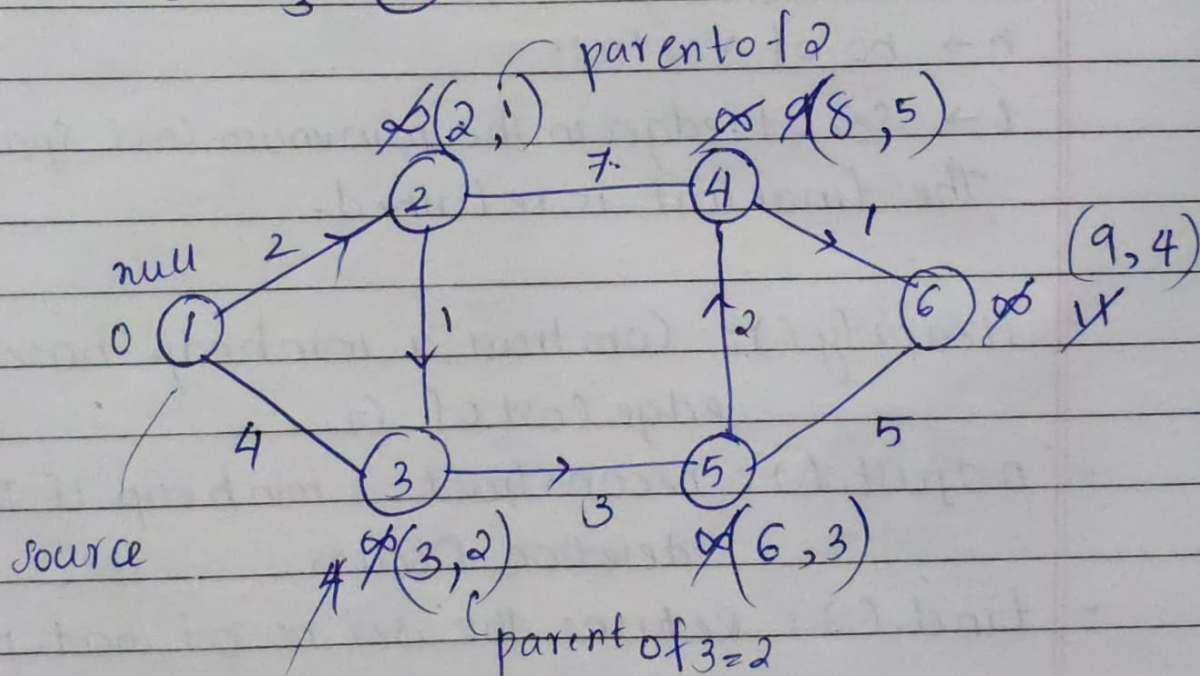
- If $J \neq k$ means vertex u and v are belongs to different set. Inclusion of (u, v) should not form cycle. add to mst.

SINGLE SOURCE SHORTEST PATH PROBLEM

Q.



$$\text{If } d(u) + c(u, v) < d(v) \\ d[v] = d[u] + c(u, v)$$



path = $1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 4 \rightarrow 6$

node	path	cost
2	1 → 2	2
3	1 → 2 → 3	3
4	1 → 2 → 3 → 5 → 4	8
5	1 → 2 → 3 → 5	6
6	1 → 2 → 3 → 5 → 4 → 6	9

Algorithm:-

Dijkstra's (G, w, s) :-

Step 1: for each vertex V in G .

1.1: Distance of $[V] = \infty$

1.2: previous $[V] = \text{null}$

2: Distance of $[s] = 0$

3: $Q = \text{set of vertices of graph } G$.

4: ^{while} ~~By~~ Q is not empty

4.1: $U = \text{vertex in } Q \text{ with minimum distance}$.

4.2: Remove U from Q .

4.3: For each neighbour ~~$[u]$~~ v of u which is still in Q .

4.3.1: $\text{alt} = \text{distance}[u] + w(u, v)$ weight

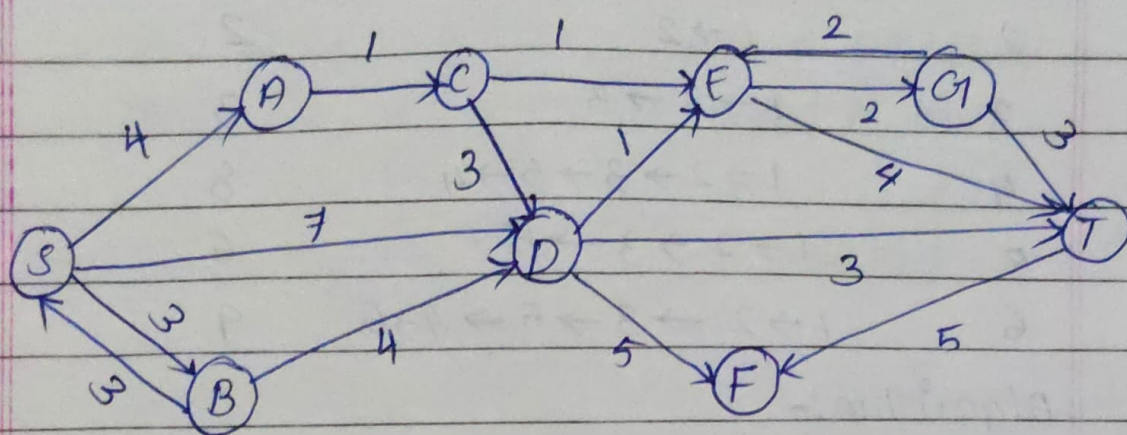
4.3.2: IF $\text{alt} < \text{distance}[v]$

4.3.2.1: $\text{distance}[v] = \text{alt}$

4.3.2.2: previous $[v] = u$.

5: Return $\text{distance}[\]$, previous $[\]$.

Q.



Find Shortest path from Source S to all other vertices in the graph.

ans:-	S	A	B	C	D	E	F	G	T
S	0	∞	∞	∞	∞	∞	∞	∞	∞
B	0	4	(3)	∞	7	∞	∞	∞	∞
A	0	(4)	7	∞	7	∞	∞	∞	∞
C				(5)	7	∞	∞	∞	∞
					7	6	∞	∞	∞
					(7)		∞	(8)	10
							∞	(8)	10
									(10)
Previous node	S	S	A	S	C	D	E	E	E

(Handwritten signature/initials)